

COMS 4720/5720: Homework 1
Due: February 6th, 11:59pm
Total Points: 40

Late submission policy. An assignment that is submitted one day late will get a penalty of 10%. An assignment that is submitted two days late will get a penalty of 20%. Assignments submitted after two days will not be graded and will get no points. For example, if an assignment is due on Friday by midnight, a submission on Saturday will be penalized by 10%, and a submission on Sunday will be penalized by 20%. A submission on Monday will get no points.

Submission format. Homework solutions must be **handwritten**. You may either write directly on a tablet or write on paper and scan your work. In both cases, the final submission must be compiled into a **single PDF file**. Typed solutions are not permitted unless you obtain prior, explicit permission from the instructor. We reserve the right **NOT** to grade homework that does not follow these requirements.

The submission file must be named as follows:

- <Your-net-id>-4720-hw1.pdf for COMS 4720
- <Your-net-id>-5720-hw1.pdf for COMS 5720.

For instance, if your netid is **asterix** and you are enrolled COMS 4720, then your submission file should be named **asterix-4720-hw1.pdf**. Each student must hand in their own assignment. If you discussed the homework or solutions with others, a list of collaborators must be included with each submission. Each of the collaborators has to write the solutions in their own words (copies are not allowed). Also, recall the AI policy:

- You may **not use AI tools to generate complete solutions** to assignments, coding problems, or any assessed work that is intended to measure your independent knowledge and skills.
- Whenever you use AI tools to assist in your assignment, you must **include a paragraph at the end of the assignment explaining what you used the AI for and include all the prompts you used to get the results**. Failure to do so is in violation of the academic honesty policies.

1. **(20 pts)** For each of the following scenarios, give a **complete problem formulation** by explicitly defining the state space, initial state, goal state(s), available actions, and transition model.

The formal problem formulation may not be unique and we only give one for reference.

- (a) **(5 pts)** There are six glass boxes in a row, each with a lock. Each of the first five boxes holds a key unlocking the next box in line; the last box holds a banana. You have the key to the first box, and you want the banana.

State space: A state is determined by whether each box is open or locked, whether the key of each box is in hand or not, and whether the agent has banana. Thus, there are totally 2^{13} possible states.

Mathematical formulation: A state $s \in S$ is defined by a tuple

$$s = (Open_1, \dots, Open_6, HasKey_1, \dots, HasKey_6, HasBanana),$$

where $Open_i \in \{0, 1\}$ indicates whether box i is open (1) or locked (0), $HasKey_i \in \{0, 1\}$ indicates whether the agent possesses the key for box i , and $HasBanana$ indicates whether the agent has banana. Thus, the state space is given by $S = \{0, 1\}^{13}$.

Initial State: The key to the first box is in hand. All the boxes are locked.

Mathematical formulation: $s_I = (0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0)$.

Goal State: Obtain the content in the 6-th box.

Mathematical formulation:

$$S_G = \{(Open_1, \dots, Open_6, HasKey_1, \dots, HasKey_6, HasBanana) \in S \mid HasBanana = 1\}$$

Actions: $A = \bigcup_i \{Unlock_i, TakeContent_i\}$

Transition Model: $Unlock_i$ opens a closed box, $TakeContent_i$ obtains the key or banana inside.

Mathematical formulation: For each natural number j , let $s_j = (O_{j,1}, \dots, O_{j,6}, K_{j,1}, \dots, K_{j,6}, B_j)$

$$\begin{aligned} f(s_1, Unlock_i) &= \begin{cases} s_2 & \text{if } K_i = 1 \\ s_1 & \text{otherwise} \end{cases} \\ f(s_1, TakeContent_i) &= \begin{cases} s_3 & \text{if } i < 6 \text{ and } O_i = 1 \\ s_4 & \text{if } i = 6 \text{ and } O_i = 1 \\ s_1 & \text{otherwise} \end{cases} \end{aligned}$$

where

- $O_{2,i} = 1$, $O_{2,k} = O_{1,k}$ for all $k \neq i$, $K_{2,l} = K_{1,l}$ for all l , and $B_2 = B_1$.
- $K_{3,i+1} = 1$, $K_{3,k} = K_{1,k}$ for all $k \neq i+1$, $O_{3,l} = O_{1,l}$ for all l , and $B_3 = B_1$.
- $B_4 = 1$, $O_{4,k} = O_{1,k}$, $K_{4,k} = K_{1,k}$ for all k .

- (b) **(5 pts)** You start with the sequence **ABABAECCCEC**, or in general any sequence made from **A**, **B**, **C**, and **E**. You can transform this sequence using the following equalities: $AC = E$, $AB = BC$, $BB = E$, and $Ex = x$ for any x . For example, **ABBC** can be transformed into **AEC**, and then **AC**, and then **E**. Your goal is to produce the sequence **E**.

State space: A state is represented by a letter sequence. The state space is infinite. Let $\Sigma = \{A, B, C, E\}$ be the alphabet. The state space $S = \Sigma^*$ is the set of all finite strings (sequences) formed by the characters in Σ .

Initial State: A given letter sequence $s_I = \text{ABABAECCCEC}$.

Goal State: The sequence **E**, i.e., $S_G = \{E\}$.

Actions: $A = \{a_1, \dots, a_4\}$, where each action corresponds to applying the following reduction $a_1 : AC = E$, $a_2 : AB = BC$, $a_3 : BB = E$, and $a_4 : Ex = x$ for all $x \in \Sigma$.

Transition Model: One sequence can be converted to another sequence if only one action is performed.

$$\begin{aligned} f(w_1ACw_2, a_1) &= w_1Ew_2 \\ f(w_1ABw_2, a_2) &= w_1BCw_2 \\ f(w_1BBw_2, a_3) &= w_1Ew_2 \\ f(Ew_2, a_4) &= w_2 \end{aligned}$$

for any $w_1, w_2 \in \Sigma^*$.

- (c) **(5 pts)** There is an $n \times n$ grid of squares, each square initially being either unpainted floor or a bottomless pit. You start standing on an unpainted floor square, and can either paint the square under you or move onto an adjacent unpainted floor square. You want the whole floor painted.

State space: A state description specifies that each of the $n \times n$ grid of squares is unpainted, painted, or a bottomless pit and the agent's position. So $S = \{0, 1, 2\}^{n \times n} \times \{1, \dots, n\}^2$. A state is $s = (M, pos) \in S$, where $M_{i,j} \in \{0, 1, 2\}$, representing a bottomless pit, unpainted floor, and painted floor, respectively and $pos = (x, y)$ represents the agent's current coordinates.

Initial State: The agent starts standing on an unpainted floor square and each of the squares is either unpainted or a bottomless pit, i.e., $s_0 = (M, (x_0, y_0))$, where $M_{x_0, y_0} = 1$ and $M_{x,y} \in \{0, 1\}$ for all x, y .

Goal State: The whole floor painted: $S_G = \{(M, pos) \in S \mid \forall x, y, M_{x,y} \neq 1\}$

Actions: Move forward $(0, 1)$; Move backward $(0, -1)$; Move left $(-1, 0)$; Move right $(1, 0)$; Paint the current square (P). So $A = \{(0, 1), (0, -1), (-1, 0), (1, 0), P\}$.

Transition Model: Agent can only move onto an adjacent unpainted floor square, or paint its current square. Let $s = (M, (x, y))$. First consider $a \neq P$ and let $(x', y') = (x, y) + a$. In this case, $a \in A(s)$ if and only if $1 \leq x' \leq n$, $1 \leq y' \leq n$, and $M_{x', y'} = 1$ and we have

$$f(s, a) = (M, (x', y'))$$

For $a = P$, we get $f(s, a) = (M', (x, y))$, where $M'_{x,y} = 2$ and $M'_{i,j} = M_{i,j}$ for all $(i, j) \neq (x, y)$.

- (d) **(5 pts)** A container ship is in port, loaded high with containers. There are 13 rows of containers, each 13 containers wide and 5 containers tall. You control a crane that can move to any location above the ship, pick up the container under it, and move it onto the dock. You want the ship unloaded.

State space: The number of containers on each of 13×13 grid squares. Specifically, $S = \{0, 1, 2, 3, 4, 5\}^{13 \times 13}$. Thus, there are totally $6^{13 \times 13}$ states. A state $s \in S$ is a matrix and we let $s_{i,j}$ denote the (i, j) element of s .

Initial State: Each grid space contains 5 shipping containers: $s_I = \mathbf{5}_{13 \times 13}$.

Goal State: The ship unloaded: $S_G = \{\mathbf{0}_{13 \times 13}\}$.

Actions: We can combine the three actions in the problem into one. Thus, use the crane to move the top container of one grid square to the dock. So $A = \{(r, c) \mid 1 \leq r, c \leq 13\}$.

Transition Model: Moving a container to the dock causes the stack on that grid have one less container. If there's no container on a grid square, we cannot perform the action here.

An action $a = (r, c) \in A(s)$ if and only if $s_{r,c} > 0$. The transition model is given by $f(s, a) = s'$, where

$$s'_{i,j} = \begin{cases} s_{i,j} - 1 & \text{if } (i, j) = (r, c) \\ s_{i,j} & \text{otherwise} \end{cases}$$

2. (20 pts) A robot is located in a maze, and your goal is to navigate it from its starting location to an exit. The maze coordinates are defined over the continuous square:

$$x \in [-1, 1], y \in [-1, 1].$$

The robot starts at position $(x, y) = (0, 0)$, facing north.

At each step, the robot may execute **exactly one** of the following actions:

- **Turn:** Rotate in place by exactly ± 90 degree, which changes its orientation to one of the four cardinal directions (north, east, south, or west).
- **Move:** Move forward along its current heading by any non-negative distance. The robot stops automatically if it encounters a wall before completing the intended distance.

The robot is considered to have exited the maze when its position reaches any point on the boundary of the square.

- (a) (5 pts) Give a **complete problem formulation** by explicitly defining the state space, initial state, goal state(s), available actions, and transition model. How large is the resulting state space?

State space: $S = [-1, 1] \times [-1, 1] \times \{0, \pi/2, \pi, 3\pi/2\}$. The state space is uncountably infinite.

Initial State: $(0, 0, \pi/2)$

Goal States: $(\{-1, 1\} \times [-1, 1] \times \{0, \pi/2, \pi, 3\pi/2\}) \cup ([-1, 1] \times \{-1, 1\} \times \{0, \pi/2, \pi, 3\pi/2\})$.

Actions: $(d, \omega) \in (\mathbb{R}_{\geq 0} \times \{0\}) \cup (\{0\} \times \{-\pi/2, \pi/2\})$, representing moving forwards any non-negative distance d and rotating by $\omega = \pm\pi/2$.

Transition Model:

$$\begin{aligned}x' &= x + d \cos \theta \\y' &= y + d \sin \theta \\\theta' &= \theta + \omega.\end{aligned}$$

- (b) **(5 pts)** Assume that the maze can be abstracted as a set of junctions connected by straight corridors, and that the robot may change direction only at junctions. Reformulate the problem under this abstraction and determine the size of the corresponding state space.

State space: Let $J \subset [-1, 1] \times [-1, 1]$ denote the finite set of junctions in the maze, where a junction is defined as a location at which two or more corridors meet. The state space is $S = J \times \{0, \pi/2, \pi, 3\pi/2\}$. Since J is finite, the state space is finite.

Initial State: The junction containing the center of the maze, with the robot facing north.

Goal States: Any state in which the robot is located at a junction that lies on an exit boundary of the maze:

$$\{(j, \theta) \mid j \in J_{\text{exit}}, \theta \in \{0, \pi/2, \pi, 3\pi/2\}\},$$

where $J_{\text{exit}} \subset J$ is the set of junctions on the maze boundary that connect to an exit.

Actions: At a junction, the robot may:

- rotate by $\pm\pi/2$, changing its orientation; or
- move forward along the corridor in its current orientation until reaching the next junction.

Transition Model: A rotation action updates the orientation by $\pm\pi/2$ while leaving the position unchanged. A move action deterministically moves the robot from its current junction to the adjacent junction reachable by following the corridor in the current direction, with orientation unchanged.

- (c) **(5 pts)** Assume that the maze contains a finite set of turning points (locations at which a change of direction may occur, including exits). From any turning point, the robot may choose one of the four cardinal directions and move directly in that direction until it reaches the next turning point. Reformulate the problem using this action set. Do we need to keep track of the robot's orientation now?

State space: Let $T \subset [-1, 1] \times [-1, 1]$ denote the finite set of turning points in the maze. A state is represented by the robot's current turning point:

$$S = T.$$

Initial state: The turning point corresponding to the center of the maze.

Goal test: The robot is at a turning point that lies on the boundary of the maze.

Actions: At a turning point $t \in T$, the robot may choose any direction $d \in \{\text{North, South, East, West}\}$ for which movement in direction d is possible. The action moves the robot directly from t to the next turning point in direction d .

Transition model: Executing the action corresponding to direction d at turning point t deterministically moves the robot to the unique turning point $t' \in T$ reached by moving in direction d .

We no longer need to keep track of the robot's orientation since it is irrelevant to predicting the outcome of our actions, and not part of the goal test. The motor system that executes this plan will need to keep track of the robot's current orientation, to know when to rotate the robot.

- (d) **(5 pts)** In our initial description of the problem we already abstracted from the real world, restricting actions and removing details. List three such simplifications we made.

State abstractions:

- i. Ignoring the height of the robot off the ground, whether it is tilted off the vertical.
- ii. The robot can face in only four directions.
- iii. Other parts of the world ignored: possibility of other robots in the maze, the weather in the Caribbean.

Action abstractions:

- i. We assumed all positions we safely accessible: the robot couldn't get stuck or damaged.
- ii. The robot can move as far as it wants, without having to recharge its batteries.
- iii. Simplified movement system: moving forwards a certain distance, rather than controlled each individual motor and watching the sensors to detect collisions.